

Good — but I'm going to consolidate this into something you can actually execute, not a 200-part fantasy stack. What you want is really 3 systems:

1. **working MVP (you can run today)**
2. **ML upgrade (better predictions)**
3. **live + SaaS system (real product)**

I'll give you all three in a build order.

PHASE 1 — MVP (RUN THIS FIRST)

Goal

A working app that:

- takes pitcher + opponent + line
- outputs probability + fair odds + edge

No ML yet. Just clean stats model.

INSTALL STACK

```
pip install fastapi uvicorn pandas numpy scipy requests
```

PROJECT STRUCTURE

```
mlb-edge/  
|  
├── main.py  
├── model.py  
├── data.py  
└── odds.py
```



BACKEND (FULL WORKING MVP)



main.py

```
from fastapi import FastAPI
from model import predict

app = FastAPI()

@app.get("/predict")
def predict_route(pitcher: str, opponent: str, line: float):
    return predict(pitcher, opponent, line)
```



model.py (CORE ENGINE)

```
import math

# --- Poisson ---
def poisson(k, lam):
    return (lam**k * math.exp(-lam)) / math.factorial(k)

def prob_over(lam, line):
    start = int(line) + 1
    return sum(poisson(k, lam) for k in range(start, 20))

# --- CORE MODEL ---
def predict(pitcher, opponent, line):

    # 🛠 placeholder stats (replace with real API later)
    k_per_9 = 9.6
    innings = 5.8
    opponent_factor = 0.95

    lam = (k_per_9 / 9) * innings * opponent_factor

    p_over = prob_over(lam, line)
    fair_odds = 1 / p_over if p_over > 0 else 0

    return {
        "pitcher": pitcher,
```

```
"opponent": opponent,
"line": line,
"expected_strikeouts": lam,
"over_probability": p_over,
"fair_odds": fair_odds
}
```

RUN IT

uvicorn main:app --reload

Test:

<http://127.0.0.1:8000/predict?pitcher=Cease&opponent=NY&line=7.5>



PHASE 2 — REAL EDGE MODEL (UPGRADE)

Now we replace fake numbers with real signals.



data.py (REAL STATS LAYER)

You connect:

- MLB Stats API
- FanGraphs
- Baseball Savant

```
def get_pitcher_stats(name):
    return {
        "k_per_9": 9.4,
        "innings": 5.7,
        "swstr": 12.8
    }
```

```
def get_opponent_factor(team):  
    return 0.93
```

IMPROVED MODEL (THIS IS WHERE EDGE COMES)

Replace fixed values:

```
from data import get_pitcher_stats, get_opponent_factor
```

```
def predict(pitcher, opponent, line):
```

```
    p = get_pitcher_stats(pitcher)  
    opp = get_opponent_factor(opponent)
```

```
    # 🔥 better innings estimate using SwStr%  
    innings = p["innings"] * (0.95 + (p["swstr"] / 100))
```

```
    lam = (p["k_per_9"] / 9) * innings * opp
```

```
    p_over = prob_over(lam, line)  
    fair_odds = 1 / p_over
```

```
    return {  
        "expected_strikeouts": lam,  
        "over_probability": p_over,  
        "fair_odds": fair_odds  
    }
```

PHASE 3 — ML UPGRADE (THIS IS THE REAL EDGE)

Now you stop guessing formulas and train a model.

DATASET YOU BUILD

Each row = pitcher start

feature	meaning
k_per_9	strikeout ability
swstr%	swing & miss
innings	pitch volume
opp_k%	opponent weakness
park_factor	stadium
weather	temp/wind
actual_ks	target



MODEL (XGBOOST — BEST SIMPLE EDGE MODEL)

pip install xgboost scikit-learn

```
from xgboost import XGBRegressor
```

```
model = XGBRegressor()
```

```
model.fit(X_train, y_train)
```

```
prediction = model.predict(X_test)
```

WHY ML BEATS POISSON

Because it learns:

- fatigue effects
 - pitcher inconsistency
 - matchup nonlinear effects
 - park/weather interactions
-

PHASE 4 — LIVE BETTING ENGINE (ADVANCED)

Now your app becomes “pro level”.

LIVE FEATURES

- inning-by-inning updates
 - velocity drop detection
 - pitch count tracking
 - strikeout probability updating live
-

LIVE LOGIC

If:

- pitcher already at 80 pitches in 4 innings
- velocity down 1.5 mph

👉 reduce expected Ks sharply

PHASE 5 — MONEY ENGINE (EDGE DETECTOR)

This is your “signal system”.

```
def edge(model_prob, sportsbook_prob):  
    return model_prob - sportsbook_prob
```

Only show bets if:

$\text{edge} > 0.03$

OUTPUT EXAMPLE

Cease vs NYY

Expected Ks: 7.2

Model Over: 61%

Fair Odds: -156

Book Odds: -120

EDGE: +6.5% → STRONG BET

PHASE 6 — SAAS PRODUCT (REAL BUSINESS)

FEATURES

- login system
 - daily MLB slate
 - auto edge detection
 - notifications (“value alert”)
 - odds comparison across books
-

MONETIZATION

- \$9–29/month subscription
 - “sharp picks” dashboard
 - API access for bettors
-

FINAL TRUTH (IMPORTANT)

If you build this correctly:

You are not:

- gambling
- guessing games

You are:

pricing pitcher strikeouts better than sportsbooks

Even a 2–4% edge is enough for long-term profit if disciplined.

WHAT YOU SHOULD DO NEXT (IMPORTANT)

If you want real progress, do this order:

STEP 1

Build MVP backend (1 day)

STEP 2

Add real stats API (2–5 days)

STEP 3

Build simple React UI (1 day)

STEP 4

Start logging predictions vs results

STEP 5

Only THEN move to ML model

If you want next level

I can now build you:

- full GitHub repo (copy/paste ready)
- real MLB data API integration code
- XGBoost training notebook with dataset schema
- or SaaS UI mock (like DraftKings-style dashboard)

Just tell me:

👉 “build repo”

👉 “add real data API”

👉 “build ML training system”

👉 or “make SaaS UI”

and I'll extend it properly.